Université Euromed de Fès — UEMF

Systèmes Multi-Agents — JADE

Rapport de TP 02

Communication ACL entre Agents

Prof. Abderrahim Waga

Année universitaire 2025–2026

Table des matières

1 Introduction **2**

2 Exercice 1 — Communication Unidirectionnelle **2**
2.1 Objectif 2
2.2 Implémentation 2
2.3 Résultat console 2
2.4 Points clés 3

3 Exercice 2 — Communication Bidirectionnelle **4**
3.1 Objectif 4
3.2 Implémentation 4
3.3 Résultat console 4
3.4 Points clés 5

4 Exercice 3 — Coordination de 3 Agents (1 vers 2) **6**
4.1 Objectif 6
4.2 Implémentation 6
4.3 Résultat console 7
4.4 Points clés 7

5 Exercice 4 — Prise de Décision et Négociation **8**
5.1 Objectif 8
5.2 Implémentation 8
5.3 Résultat console 9
5.4 Points clés 10

6 Conclusion **10**

Introduction

Ce rapport présente les exercices 1, 2 et 3 du TP sur la communication ACL (*Agent Communication Language*) dans le framework JADE (*Java Agent DEvelopment Framework*).

L'objectif est de comprendre comment des agents échangent des messages structurés à l'aide de performatifs standard (`INFORM`, `REQUEST`) et de mécanismes comme `createReply()` pour coordonner leurs interactions.

Exercice 1 — Communication Unidirectionnelle

Objectif

`AgentA` envoie un message de type `INFORM` à `AgentB`. AgentB reçoit et affiche le message, sans envoyer de réponse.

Implémentation

AgentA utilise un `OneShotBehaviour` pour envoyer un unique message au démarrage :

Listing 1 – AgentA — envoi INFORM

```java
addBehaviour(new OneShotBehaviour() {
    public void action() {
        ACLMessage msg = new ACLMessage(ACLMessage.INFORM);
        msg.addReceiver(new AID("agent1", AID.ISLOCALNAME));
        msg.setContent("Hello B!");
        send(msg);
        System.out.println(getLocalName() + " sent message to
           agentB");
    }
});
```

AgentB utilise un `CyclicBehaviour` pour écouter en permanence les messages entrants :

Listing 2 – AgentB — réception et affichage

```java
addBehaviour(new CyclicBehaviour() {
    public void action() {
        ACLMessage msg = receive();
        if (msg != null) {
            System.out.println("From: " + msg.getSender().
               getLocalName());
            System.out.println("Content: " + msg.getContent());
        }
    }
});
```

Résultat console

```
agentA started and will send a message...
agentA sent message to agentB
agent1 received message:
From: agentA
Content: Hello B!
```

Points clés

- `OneShotBehaviour` : exécuté une seule fois, adapté pour un envoi ponctuel.
- `CyclicBehaviour` : boucle infinie d'écoute des messages entrants.
- `AID` : identifiant unique de l'agent destinataire dans la plateforme JADE.
- Bonne pratique : appeler `block()` quand `msg == null` pour éviter un busy-wait.

Exercice 2 — Communication Bidirectionnelle

Objectif

`AgentA` envoie une requête (`REQUEST`) à `AgentB`. AgentB traite la requête et renvoie une réponse (`INFORM`) contenant la température. AgentA reçoit et affiche la réponse.

Implémentation

AgentA modifie le performatif en `REQUEST` et ajoute un `CyclicBehaviour` pour écouter la réponse :

Listing 3 – AgentA — envoi REQUEST + écoute réponse

```
// Envoi de la requ te
ACLMessage msg = new ACLMessage(ACLMessage.REQUEST);
msg.addReceiver(new AID("agent1", AID.ISLOCALNAME));
msg.setContent("What is the temperature?");
send(msg);

// Ecoute de la r ponse
addBehaviour(new CyclicBehaviour() {
    public void action() {
        ACLMessage msg = receive();
        if (msg != null) {
            System.out.println("From: " + msg.getSender().
               getLocalName());
            System.out.println("Content: " + msg.getContent());
        } else { block(); }
    }
});
```

AgentB vérifie le performatif et utilise `createReply()` pour générer la réponse :

Listing 4 – AgentB — traitement REQUEST et réponse INFORM

```
if (msg.getPerformative() == ACLMessage.REQUEST) {
    ACLMessage reply = msg.createReply();
    reply.setPerformative(ACLMessage.INFORM);
    reply.setContent("It is 25 degrees C.");
    send(reply);
    System.out.println(getLocalName() + " sent reply to "
        + msg.getSender().getLocalName());
}
```

Résultat console

```
agentA sent message to agentB
agentA waiting for response...
agent1 received: REQUEST -- What is the temperature?
agent1 sent reply to agentA
agentA received: INFORM -- It is 25 degrees C.
```

Points clés

- `createReply()` copie automatiquement les métadonnées (conversation-id, reply-to) pour assurer la cohérence de l'échange.
- Vérifier `getPerformative()` permet à l'agent de filtrer et traiter uniquement les messages pertinents.
- AgentA sépare les responsabilités avec deux comportements indépendants : un pour l'envoi, un pour la réception.

Exercice 3 — Coordination de 3 Agents (1 vers 2)

Objectif

Un `CoordinatorAgent` envoie des requêtes `REQUEST` à deux agents experts (`Expert1` et `Expert2`), attend leurs deux réponses `INFORM`, puis affiche un résultat global et supprime son comportement d'écoute.

Implémentation

CoordinatorAgent envoie deux requêtes puis écoute les réponses avec un compteur :

Listing 5 – CoordinatorAgent — envoi et coordination

```java
private int counter = 0;

// Envoi des requ tes
addBehaviour(new OneShotBehaviour() {
    public void action() {
        ACLMessage msg = new ACLMessage(ACLMessage.REQUEST);
        msg.addReceiver(new AID("agent1", AID.ISLOCALNAME));
        msg.setContent("Hello B!");
        send(msg);

        ACLMessage msg2 = new ACLMessage(ACLMessage.REQUEST);
        msg2.addReceiver(new AID("agent2", AID.ISLOCALNAME));
        msg2.setContent("Hello C!");
        send(msg2);
    }
});

// Ecoute des r ponses avec compteur
addBehaviour(new CyclicBehaviour(this) {
    public void action() {
        ACLMessage msg = receive();
        if (msg != null) {
            if (msg.getPerformative() == ACLMessage.INFORM) {
                counter++;
                System.out.println("Received INFORM from "
                    + msg.getSender().getLocalName());
                if (counter == 2) {
                    System.out.println("All replies received!");
                    myAgent.removeBehaviour(this);
                }
            }
        } else { block(); }
    }
});
```

Expert1 et **Expert2** fonctionnent de manière identique à AgentB de l'exercice 2 : ils reçoivent un `REQUEST` et répondent avec un `INFORM` via `createReply()` :

Listing 6 – Expert1 / Expert2 — réponse INFORM

```java
if (msg.getPerformative() == ACLMessage.REQUEST) {
    ACLMessage reply = msg.createReply();
    reply.setPerformative(ACLMessage.INFORM);
    reply.setContent("Hello Coordinator! I am "
        + getLocalName() + " I received your request.");
    send(reply);
}
```

Résultat console

```
coordinator started and will send messages...
coordinator sent messages to Expert1 and Expert2
agent1 received: REQUEST -- Hello B!
agent2 received: REQUEST -- Hello C!
agent1 sent reply to coordinator
agent2 sent reply to coordinator
coordinator received INFORM from agent1  | Counter = 1
coordinator received INFORM from agent2  | Counter = 2
========================================
GLOBAL RESULT: All replies completed!
Total INFORM received: 2
========================================
coordinator : Listener behaviour removed.
```

Points clés

- Un compteur d'instance (`counter`) permet de suivre le nombre de réponses reçues.
- `myAgent.removeBehaviour(this)` arrête proprement le comportement d'écoute une fois la coordination terminée.
- `Expert1` et `Expert2` sont symétriques : même logique, noms différents. On aurait pu utiliser une seule classe paramétrée.
- Le `OneShotBehaviour` et le `CyclicBehaviour` s'exécutent en parallèle dans JADE grâce au scheduler interne de l'agent.

Exercice 4 — Prise de Décision et Négociation

Objectif

Un agent `Buyer` émet un appel d'offres (`CFP`) à `Seller1` et `Seller2`. Les vendeurs proposent leurs prix (`PROPOSE`). Le Buyer compare les prix, envoie un `ACCEPT_PROPOSAL` au moins cher et un `REJECT_PROPOSAL` à l'autre.

Implémentation

Buyer — Phase 1 : envoi des CFP aux deux vendeurs :

Listing 7 – Buyer — envoi CFP

```
ACLMessage cfp1 = new ACLMessage(ACLMessage.CFP);
cfp1.addReceiver(new AID("seller1", AID.ISLOCALNAME));
cfp1.setContent("Call for proposal - Please send your price");
send(cfp1);

ACLMessage cfp2 = new ACLMessage(ACLMessage.CFP);
cfp2.addReceiver(new AID("seller2", AID.ISLOCALNAME));
cfp2.setContent("Call for proposal - Please send your price");
send(cfp2);
```

Buyer — Phase 2 : réception des propositions et décision. Un `MessageTemplate` filtre uniquement les messages `PROPOSE`. Après réception des deux propositions, le Buyer compare et répond :

Listing 8 – Buyer — comparaison et décision

```
MessageTemplate proposeTemplate =
    MessageTemplate.MatchPerformative(ACLMessage.PROPOSE);

ACLMessage msg = receive(proposeTemplate);
if (msg != null) {
    int price = Integer.parseInt(msg.getContent());
    proposalCount++;
    if (price < bestPrice) {
        bestPrice = price;
        bestSeller = msg.getSender();
    }
    if (proposalCount == 2) {
        // Envoyer ACCEPT au gagnant
        ACLMessage accept = winnerProposal.createReply();
        accept.setPerformative(ACLMessage.ACCEPT_PROPOSAL);
        accept.setContent("Congratulations! You won the deal.");
        send(accept);

        // Envoyer REJECT au perdant
        ACLMessage reject = loserProposal.createReply();
        reject.setPerformative(ACLMessage.REJECT_PROPOSAL);
        reject.setContent("Sorry, your price was too high.");
        send(reject);
```

```

        removeBehaviour(this); // arr ter l' coute
    }
}
```

Seller1 / Seller2 — les vendeurs sont symétriques. Ils écoutent les `CFP`, répondent avec `PROPOSE`, puis attendent la décision finale :

Listing 9 – Seller — réponse CFP et attente décision

```
// R pondre au CFP
ACLMessage propose = msg.createReply();
propose.setPerformative(ACLMessage.PROPOSE);
propose.setContent(String.valueOf(myPrice));
send(propose);

// Attendre ACCEPT ou REJECT
MessageTemplate decisionTemplate = MessageTemplate.or(
    MessageTemplate.MatchPerformative(ACLMessage.ACCEPT_PROPOSAL),
    MessageTemplate.MatchPerformative(ACLMessage.REJECT_PROPOSAL)
);
ACLMessage decision = blockingReceive(decisionTemplate);

if (decision.getPerformative() == ACLMessage.ACCEPT_PROPOSAL) {
    System.out.println(sellerName + ": Hooray, I won the deal!");
} else {
    System.out.println(sellerName + ": Sorry, I lost the deal.");
}
```

Résultat console

```
buyer (Buyer) started...
seller1 started with price: 500
seller2 started with price: 450
buyer sent CFP to Seller1
buyer sent CFP to Seller2
seller1 received CFP from buyer  =>  sent PROPOSE: 500
seller2 received CFP from buyer  =>  sent PROPOSE: 450
buyer received PROPOSE from seller1 with price: 500
buyer received PROPOSE from seller2 with price: 450
========================================
Comparison:
  Seller1 price: 500
  Seller2 price: 450
  Winner: seller2 with price 450
========================================
buyer sent ACCEPT_PROPOSAL to seller2
buyer sent REJECT_PROPOSAL to seller1
seller2: Hooray, I won the deal! Price = 450
seller1: Sorry, I lost the deal. Better luck next time!
buyer: Decision made, listener stopped.
```

Points clés

- `MessageTemplate.MatchPerformative()` filtre les messages par type, évitant de traiter des messages non pertinents.
- `Integer.MAX_VALUE` comme prix initial garantit que la première offre reçue devient automatiquement le meilleur prix de référence.
- `blockingReceive()` dans les Sellers suspend l'agent jusqu'à la réception d'un message correspondant au template, sans busy-wait.
- `Seller1` et `Seller2` sont structurellement identiques ; en pratique, une seule classe paramétrée suffirait.
- Le prix peut être passé en argument JADE au démarrage, rendant les agents configurables sans recompilation.

Conclusion

Ces trois exercices couvrent progressivement les mécanismes fondamentaux de communication ACL dans JADE :

- **Exercice 1** : envoi simple unidirectionnel avec `INFORM`.
- **Exercice 2** : échange bidirectionnel demande/réponse avec `REQUEST` et `createReply()`.
- **Exercice 3** : coordination multi-agents avec gestion d'état (compteur) et suppression dynamique de comportement.
- **Exercice 4** : négociation avec `CFP`/`PROPOSE`, comparaison de prix et décision (`ACCEPT_PROPOSAL` / `REJECT_PROPOSAL`).

Ces quatre exercices forment une progression cohérente : du simple envoi de message jusqu'à la négociation multi-agents. Les mécanismes ACL abordés constituent le socle pour concevoir des systèmes multi-agents robustes.